宠物服务模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 服务表(service)

字段名	数据类型	描述	约束条件
id	BIGINT	服务ID	主键，自增
name	VARCHAR(100)	服务名称	非空
category_id	BIGINT	分类ID	外键关联service_category表
price	DECIMAL(10,2)	价格	非空
description	TEXT	描述	可选
duration	INTEGER	时长(分钟)	非空
status	INTEGER	状态(0:停用,1:启用)	非空，默认1
create_time	DATETIME	创建时间	自动生成
update_time	DATETIME	更新时间	自动更新

Service类：对应服务表，包含服务基本信息
【springboot/src/main/java/org/example/springboot/entity/Service.java】

```
@Data
@TableName("service")
@Schema(description = "服务实体")
public class Service {

    @TableId(type = IdType.AUTO)
    @Schema(description = "服务ID")
    private Long id;
```

半夜撕代码版权所有

```java
    @Schema(description = "服务名称")
    private String name;

    @Schema(description = "分类ID")
    private Long categoryId;

    @Schema(description = "价格")
    private BigDecimal price;

    @Schema(description = "描述")
    private String description;

    @Schema(description = "时长(分钟)")
    private Integer duration;

    @Schema(description = "状态(0:停用,1:启用)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;
}
```

1.1.2 服务分类表(service_category)

字段名	数据类型	描述	约束条件
id	BIGINT	分类ID	主键，自增
name	VARCHAR(50)	分类名称	非空，唯一
description	TEXT	分类描述	可选
icon	VARCHAR(255)	分类图标	可选
sort	INTEGER	排序	默认0
status	INTEGER	状态(0:禁用,1:正常)	非空，默认1
create_time	DATETIME	创建时间	自动生成
update_time	DATETIME	更新时间	自动更新

ServiceCategory类：对应服务分类表，包含分类基本信息
【springboot/src/main/java/org/example/springboot/entity/ServiceCategory.java】

半夜撕代码版权所有

```java
@Data
@TableName("service_category")
@Schema(description = "服务分类实体")
public class ServiceCategory {

    @TableId(type = IdType.AUTO)
    @Schema(description = "分类ID")
    private Long id;

    @Schema(description = "分类名称")
    private String name;

    @Schema(description = "分类描述")
    private String description;

    @Schema(description = "分类图标")
    private String icon;

    @Schema(description = "排序")
    private Integer sort;

    @Schema(description = "状态(0:禁用,1:正常)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;
}
```

1.1.3 服务预约表(service_appointment)

字段名	数据类型	描述	约束条件
id	BIGINT	预约ID	主键，自增
user_id	BIGINT	用户ID	外键关联user表
service_id	BIGINT	服务ID	外键关联service表
appointment_time	DATETIME	预约时间	非空
status	VARCHAR(20)	状态(已预约/已确认/已完成/已取消)	非空，默认"已预约"

半夜撕代码版权所有

字段名	数据类型	描述	约束条件
pet_name	VARCHAR(50)	宠物名称	非空
requirements	TEXT	特殊要求	可选
contact_phone	VARCHAR(20)	联系电话	非空
create_time	DATETIME	创建时间	自动生成
update_time	DATETIME	更新时间	自动更新

ServiceAppointment类：对应服务预约表，包含预约基本信息
【springboot/src/main/java/org/example/springboot/entity/ServiceAppointment.java】

```
@Data
@TableName("service_appointment")
@Schema(description = "服务预约实体")
public class ServiceAppointment {

    @TableId(type = IdType.AUTO)
    @Schema(description = "预约ID")
    private Long id;

    @Schema(description = "用户ID")
    private Long userId;

    @Schema(description = "服务ID")
    private Long serviceId;

    @Schema(description = "预约时间")
    private LocalDateTime appointmentTime;

    @Schema(description = "状态(已预约/已确认/已完成/已取消)")
    private String status;

    @Schema(description = "宠物名称")
    private String petName;

    @Schema(description = "特殊要求")
    private String requirements;

    @Schema(description = "联系电话")
    private String contactPhone;

    @Schema(description = "创建时间")
```

半夜撕代码版权所有

```
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;
}
```

1.2 核心属性说明

- **服务状态管理**：服务和服务分类都有状态字段，用于控制是否可用
- **预约状态流转**：预约状态包括"已预约"、"已确认"、"已完成"、"已取消"，表示预约的不同阶段
- **时长管理**：服务设置了时长字段，以分钟为单位，用于表示服务所需时间
- **排序功能**：服务分类具有排序字段，用于前端展示的顺序控制

二、各功能详细讲解

2.1 服务管理功能

2.1.1 功能概述

服务管理功能允许管理员创建、编辑、删除和查询宠物服务项目，包括设置服务名称、分类、价格、时长和状态等信息。管理员可以通过后台界面对服务进行全面管理，确保服务信息的准确性和可用性。

2.1.2 详细讲解功能实现流程

服务管理功能主要在后台管理界面实现。管理员在ServiceManagement.vue页面中可以查看所有服务列表，通过表单筛选服务，点击"添加"按钮打开对话框填写服务信息，提交后前端通过request.post("/service")发送请求到后端ServiceController的addService方法，该方法调用ServiceService的addService方法进行数据验证并保存到数据库。编辑服务时，管理员点击编辑按钮，填写修改后的信息，提交后前端通过request.put("/service/{id}")发送请求到后端updateService方法，更新服务信息。

2.1.3 关键代码讲解

1. 后端服务控制器
   【springboot/src/main/java/org/example/springboot/controller/ServiceController.java】

```
@Operation(summary = "添加服务")
@PostMapping
public Result<?> addService(@RequestBody Service service) {
    LOGGER.info("添加服务: {}", service);
    boolean success = serviceService.addService(service);
    return success ? Result.success(service.getId()) : Result.error("添加服务失败");
```

半夜撕代码版权所有

```
    }

    @Operation(summary = "更新服务")
    @PutMapping("/{id}")
    public Result<?> updateService(@PathVariable Long id, @RequestBody
    Service service) {
        LOGGER.info("更新服务: id={}, service={}", id, service);
        service.setId(id);
        boolean success = serviceService.updateService(service);
        return success ? Result.success() : Result.error("更新服务失败");
    }
```

这段代码定义了添加和更新服务的API接口。添加服务接口接收Service对象并调用serviceService.addService方法保存数据；更新服务接口接收服务ID和Service对象，设置ID后调用serviceService.updateService方法更新数据。

2. 后端服务业务逻辑
   【springboot/src/main/java/org/example/springboot/service/ServiceService.java】

```
    public boolean addService(Service service) {
        if (service == null) {
            throw new ServiceException("服务信息不能为空");
        }

        // 设置初始状态为启用
        if (service.getStatus() == null) {
            service.setStatus(1);
        }

        validateService(service);

        int result = serviceMapper.insert(service);
        return result > 0;
    }

    private void validateService(Service service) {
        if (StringUtils.isBlank(service.getName())) {
            throw new ServiceException("服务名称不能为空");
        }

        if (service.getCategoryId() == null) {
            throw new ServiceException("服务分类不能为空");
        }

        if (service.getPrice() == null) {
            throw new ServiceException("服务价格不能为空");
        }
```

半夜撕代码版权所有

```
        if (service.getPrice().doubleValue() < 0) {
            throw new ServiceException("服务价格不能为负数");
        }

        if (service.getDuration() == null) {
            throw new ServiceException("服务时长不能为空");
        }

        if (service.getDuration() <= 0) {
            throw new ServiceException("服务时长必须大于0");
        }
    }
```

这段代码实现了添加服务的业务逻辑，包括数据验证和保存。validateService方法对服务信息进行全面验证，确保必填字段不为空且满足业务规则，如价格不能为负数，时长必须大于0等。

3. 前端服务管理界面
【vue3/src/views/backend/service/ServiceManagement.vue】

```
const submitForm = async () => {
  if (!formRef.value) return

  await formRef.value.validate(async (valid) => {
    if (valid) {
      submitting.value = true
      try {
        const isEdit = !!form.value.id
        const url = isEdit ? `/service/${form.value.id}` : '/service'
        const method = isEdit ? 'put' : 'post'

        await request[method](url, form.value, {
          onSuccess: () => {
            ElMessage.success(t(`backend.service.message.${isEdit ?
'updateSuccess' : 'addSuccess'}`))
            dialogVisible.value = false
            fetchData()
          }
        })
      } catch (error) {
        console.error('提交服务表单失败:', error)
      } finally {
        submitting.value = false
      }
    }
  })
}
```

这段代码实现了前端提交服务表单的逻辑。根据是否有ID判断是新增还是编辑，选择对应的HTTP方法和URL，提交表单数据到后端。成功后关闭对话框并刷新数据列表。

半夜撕代码版权所有

2.1.4 功能实现细节

- **表单验证**：前端和后端都实现了严格的表单验证，确保数据的完整性和有效性
- **状态管理**：服务可以设置为启用或停用状态，停用状态的服务不会在前端展示
- **分类关联**：服务必须关联到一个服务分类，确保服务的分类管理
- **价格和时长**：服务必须设置价格和时长，用于用户了解服务信息

2.2 服务分类管理功能

2.2.1 功能概述

服务分类管理功能允许管理员对宠物服务的分类进行管理，包括创建、编辑、删除分类，以及设置分类的状态和排序。通过合理的分类管理，可以帮助用户更快地找到所需的服务。

2.2.2 详细讲解功能实现流程

管理员在ServiceCategoryManagement.vue页面中管理服务分类，可以查看所有分类列表，添加新分类时填写分类名称、描述和图标等信息，提交后前端通过request.post("/service/category")发送请求到后端ServiceCategoryController的addCategory方法，该方法调用ServiceCategoryService的addCategory方法验证并保存分类信息。编辑分类时，前端通过request.put("/service/category/{id}")发送请求到后端updateCategory方法更新分类信息。

2.2.3 关键代码讲解

1. 后端分类控制器【springboot/src/main/java/org/example/springboot/controller/ServiceCategoryController.java】

```
@Operation(summary = "获取所有分类列表")
@GetMapping("/list")
public Result<?> getAllCategories() {
    LOGGER.info("获取所有分类列表");
    List<ServiceCategory> list =
serviceCategoryService.getAllCategories();
    return Result.success(list);
}

@Operation(summary = "获取所有启用状态的分类列表")
@GetMapping("/enabled")
public Result<?> getEnabledCategories() {
    LOGGER.info("获取所有启用状态的分类列表");
    List<ServiceCategory> list =
serviceCategoryService.getEnabledCategories();
    return Result.success(list);
}
```

半夜撕代码版权所有

这段代码定义了获取分类列表的API接口。getAllCategories方法获取所有分类，getEnabledCategories方法只获取启用状态的分类，用于前端展示。

2. 后端分类业务逻辑
【springboot/src/main/java/org/example/springboot/service/ServiceCategoryService.java】

```
public boolean addCategory(ServiceCategory category) {
    if (category == null) {
        throw new ServiceException("分类信息不能为空");
    }

    // 设置初始状态为启用
    if (category.getStatus() == null) {
        category.setStatus(1);
    }

    validateCategory(category);

    // 检查是否有同名分类
    checkDuplicateCategoryName(category.getName(), null);

    int result = serviceCategoryMapper.insert(category);
    return result > 0;
}

private void checkDuplicateCategoryName(String name, Long id) {
    LambdaQueryWrapper<ServiceCategory> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(ServiceCategory::getName, name);
    if (id != null) {
        queryWrapper.ne(ServiceCategory::getId, id);
    }

    Long count = serviceCategoryMapper.selectCount(queryWrapper);
    if (count > 0) {
        throw new ServiceException("分类名称已存在");
    }
}
```

这段代码实现了添加分类的业务逻辑，包括数据验证和保存。特别是checkDuplicateCategoryName方法检查分类名称的唯一性，防止创建重复的分类名称。

2.2.4 功能实现细节

- **名称唯一性**：系统确保分类名称的唯一性，防止出现重复分类
- **状态管理**：分类可以设置为启用或禁用状态，禁用状态的分类不会在前端展示
- **排序功能**：分类具有排序字段，用于控制在前端展示的顺序

半夜撕代码版权所有

- **关联检查**：删除分类前会检查是否有服务关联到该分类，防止误删除

2.3 服务预约功能

2.3.1 功能概述

服务预约功能允许用户在前端浏览服务列表，查看服务详情，并进行在线预约。用户可以选择服务、填写预约信息，提交后系统会记录预约信息并等待管理员确认。用户可以查看自己的预约历史和状态。

2.3.2 详细讲解功能实现流程

用户在ServiceList.vue页面浏览服务列表，点击"预约"按钮进入ServiceBooking.vue页面，填写宠物名称、预约时间、联系电话等信息，点击提交后前端通过request.post("/service/appointment")发送请求到后端ServiceAppointmentController的createAppointment方法，该方法调用ServiceAppointmentService的createAppointment方法验证并保存预约信息。用户可以在MyAppointments.vue页面查看自己的预约记录和状态，也可以取消预约。

2.3.3 关键代码讲解

1. 后端预约控制器【springboot/src/main/java/org/example/springboot/controller/ServiceAppointmentController.java】

```
@Operation(summary = "创建预约")
@PostMapping
public Result<?> createAppointment(@RequestBody ServiceAppointment appointment) {
    LOGGER.info("创建预约: {}", appointment);
    Long id = appointmentService.createAppointment(appointment);
    return Result.success(id);
}

@Operation(summary = "取消预约（用户端）")
@PutMapping("/{id}/cancel")
public Result<?> cancelAppointment(@PathVariable Long id, @RequestBody ServiceAppointmentDTO dto) {
    LOGGER.info("取消预约: id={}, userId={}", id, dto.getUserId());
    if (dto.getUserId() == null) {
        return Result.error("用户ID不能为空");
    }

    boolean success = appointmentService.cancelAppointment(id, dto.getUserId());
    return success ? Result.success() : Result.error("取消预约失败");
}
```

这段代码定义了创建和取消预约的API接口。创建预约接口接收ServiceAppointment对象并调用appointmentService.createAppointment方法保存数据；取消预约接口接收预约ID和用户ID，调用appointmentService.cancelAppointment方法取消预约。

2. 前端预约表单
【vue3/src/views/frontend/service/ServiceBooking.vue】

```
const submitForm = async () => {
  if (!formRef.value) return

  await formRef.value.validate(async (valid) => {
    if (valid) {
      if (!userStore.isLoggedIn) {
        ElMessageBox.confirm(
          t('service.booking.loginRequired'),
          t('common.tip'),
          {
            confirmButtonText: t('auth.login'),
            cancelButtonText: t('common.cancel'),
            type: 'warning'
          }
        ).then(() => {
          router.push({
            name: 'Login',
            query: { redirect: router.currentRoute.value.fullPath }
          })
        })
        return
      }

      submitting.value = true
      try {
        const appointmentData = {
          userId: userStore.userInfo.id,
          serviceId: serviceId,
          petName: form.value.petName,
          appointmentTime: form.value.appointmentTime,
          contactPhone: form.value.contactPhone,
          requirements: form.value.requirements
        }

        await request.post('/service/appointment', appointmentData, {
          onSuccess: () => {
            ElMessage.success(t('service.booking.success'))
            showSuccessDialog.value = true
          }
        })
      } catch (error) {
        console.error('提交预约失败:', error)
```

```
        } finally {
          submitting.value = false
        }
      }
    })
  }
```

这段代码实现了前端提交预约表单的逻辑。首先验证用户是否登录，未登录则提示登录；然后验证表单数据，构造预约数据对象并提交到后端。成功后显示成功对话框。

2.3.4 功能实现细节

- **登录验证**：预约前会检查用户是否已登录，未登录会提示登录
- **表单验证**：前端和后端都实现了严格的表单验证，确保数据的完整性
- **时间限制**：预约时间不能早于当前时间，确保预约的有效性
- **状态流转**：预约状态从"已预约"开始，可以流转到"已确认"、"已完成"或"已取消"

2.4 预约管理功能

2.4.1 功能概述

预约管理功能允许管理员查看和处理用户的服务预约请求，包括确认预约、完成预约、取消预约等操作。管理员可以通过后台界面对预约进行全面管理，确保服务的顺利进行。

2.4.2 详细讲解功能实现流程

管理员在AppointmentManagement.vue页面中可以查看所有预约记录，通过表单筛选预约，点击"确认"、"完成"或"取消"按钮对预约进行处理。处理预约时，前端通过request.put("/service/appointment/{id}/status")发送请求到后端ServiceAppointmentController的updateAppointmentStatus方法，该方法调用ServiceAppointmentService的updateAppointmentStatus方法更新预约状态。

2.4.3 关键代码讲解

1. 后端预约状态更新
   【springboot/src/main/java/org/example/springboot/service/ServiceAppointmentService.java】

```
public boolean updateAppointmentStatus(Long id, ServiceAppointmentDTO
dto) {
    if (id == null) {
        throw new ServiceException("预约ID不能为空");
    }

    if (StringUtils.isBlank(dto.getStatus())) {
        throw new ServiceException("状态不能为空");
    }
```

```
        // 检查预约是否存在
        ServiceAppointment appointment = appointmentMapper.selectById(id);
        if (appointment == null) {
            throw new ServiceException("预约不存在");
        }

        // 检查状态变更是否合法
        if (!isValidStatusChange(appointment.getStatus(), dto.getStatus()))
    {
            throw new ServiceException("状态变更不合法");
        }

        // 更新状态
        appointment.setStatus(dto.getStatus());

        int result = appointmentMapper.updateById(appointment);
        return result > 0;
    }

    private boolean isValidStatusChange(String currentStatus, String
    newStatus) {
        // 定义状态流转规则
        if ("已预约".equals(currentStatus)) {
            return "已确认".equals(newStatus) || "已取消".equals(newStatus);
        } else if ("已确认".equals(currentStatus)) {
            return "已完成".equals(newStatus) || "已取消".equals(newStatus);
        }

        // 已完成和已取消状态不允许再变更
        return false;
    }
```

这段代码实现了更新预约状态的业务逻辑。特别是isValidStatusChange方法定义了预约状态的流转规则，确保状态变更的合法性，如"已预约"状态只能变更为"已确认"或"已取消"，"已完成"和"已取消"状态不允许再变更。

2.4.4 功能实现细节

- **状态流转控制**：系统严格控制预约状态的流转，确保状态变更的合法性
- **多条件查询**：管理员可以通过多种条件筛选预约记录，如用户ID、宠物名称、状态等
- **分页展示**：预约记录通过分页方式展示，提高界面性能和用户体验
- **权限控制**：只有管理员可以确认和完成预约，普通用户只能取消自己的预约

半夜撕代码版权所有

2.5 与其它模块交互

宠物服务模块与用户模块紧密关联，预约服务时需要用户登录，并记录用户ID。同时，服务模块也可能与支付模块关联，用户预约服务后可能需要支付相应费用。此外，服务模块还可能与评价模块关联，用户在服务完成后可以对服务进行评价。

三、总结

3.1 服务管理功能实现

宠物服务管理功能通过ServiceController和ServiceService实现，管理员在ServiceManagement.vue界面中可以创建、编辑和管理服务项目。添加服务时，前端通过表单收集服务信息，调用request.post("/service", form.value)将数据提交至后端。后端ServiceController.addService()接口接收请求并调用serviceService.addService(service)方法，该方法使用validateService()进行数据验证，确保服务名称、分类、价格和时长等必填字段的有效性，然后通过serviceMapper.insert(service)保存数据。服务状态管理通过updateServiceStatus()方法实现，可以启用或停用服务。

3.2 服务分类管理功能实现

服务分类管理功能通过ServiceCategoryController和ServiceCategoryService实现，管理员可以在后台创建和管理服务分类。添加分类时，前端提交表单数据到/service/category接口，后端ServiceCategoryController.addCategory()方法接收请求并调用serviceCategoryService.addCategory(category)处理。该方法使用checkDuplicateCategoryName()确保分类名称唯一性，防止出现重复分类。分类列表通过getAllCategories()和getEnabledCategories()方法获取，后者只返回启用状态的分类，用于前端服务展示。分类排序通过sort字段控制，影响前端展示顺序。

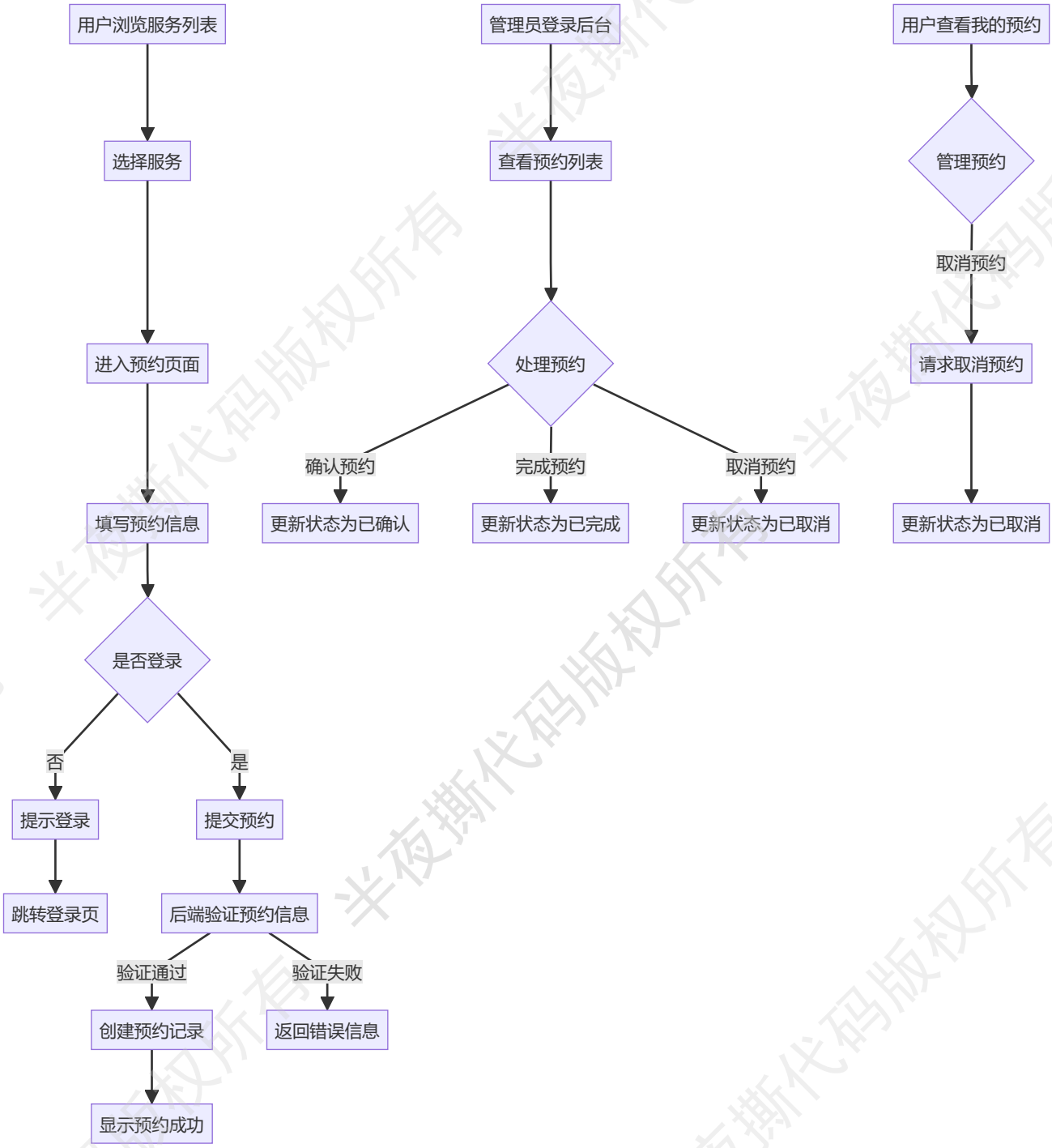
3.3 服务预约功能实现

服务预约功能允许用户在前端预约宠物服务。用户浏览服务列表(ServiceList.vue)后，点击预约进入预约页面(ServiceBooking.vue)，填写宠物名称、预约时间等信息。提交时，前端验证用户登录状态，然后通过request.post("/service/appointment", appointmentData)发送预约请求。后端ServiceAppointmentController.createAppointment()方法接收请求并调用appointmentService.createAppointment(appointment)处理，该方法验证预约信息的有效性，包括检查用户和服务存在性、预约时间合法性等。用户可以在MyAppointments.vue页面查看和管理自己的预约，通过cancelAppointment()方法取消预约。

3.4 预约管理功能实现

预约管理功能通过AppointmentManagement.vue界面实现，管理员可以查看和处理所有预约请求。管理员可以通过多种条件筛选预约记录，如用户ID、服务名称、状态等。处理预约时，前端调用request.put("/service/appointment/{id}/status", dto)发送请求，后端ServiceAppointmentController.updateAppointmentStatus()方法接收请求并调用appointmentService.updateAppointmentStatus(id, dto)处理。该方法使用

isValidStatusChange()确保状态变更的合法性，如"已预约"只能变为"已确认"或"已取消"，"已完成"状态不能再变更。系统通过这种方式严格控制预约状态的流转过程。



半夜撕代码版权所有